

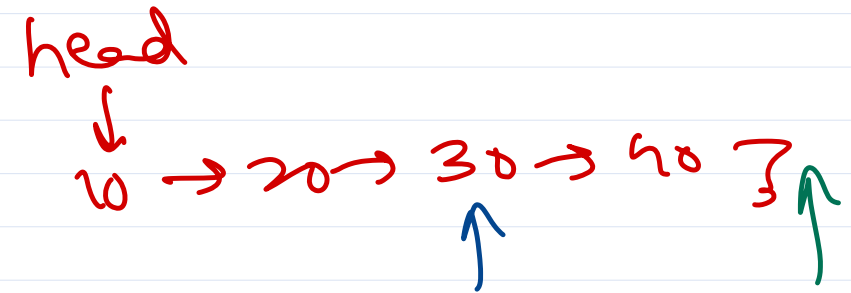
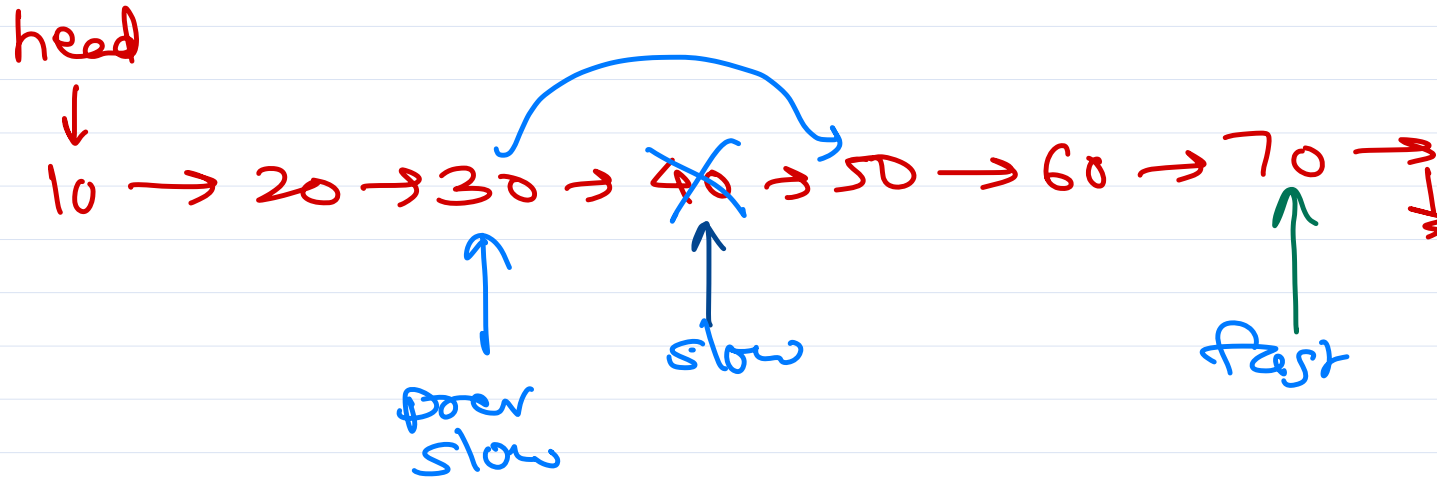


Sunbeam Infotech

Exploring new ideas, Reaching new heights!

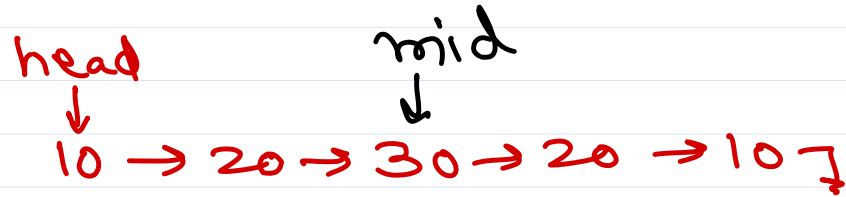


Delete Middle



```
fast = head;
slow = head;
while (fast != null && fast.next != null) {
    prevslow = slow;
    slow = slow.next;
    fast = fast.next.next;
}
if (slow != head)
    prevslow.next = slow.next;
else
    head = null;
```

Check if list is palindrome



Approach 2: using stack:

```
Stack<Integer> s = new Stack();
trav = head;
while (trav != null) {
    s.push(trav.val);
    trav = trav.next;
}
trav = head;
while (trav != null) {
    v = s.pop();
    if (trav.val != v)
        return false;
    trav = trav.next;
}
return true;
```

$itr = 2 * n$

approach 1:

- ① create a new list as copy of existing list. $itr = 3 * n$
- ② reverse that new list.
- ③ Compare original list with new list. if same then palindrome, else not.

approach 3:

- ① Split list in two equal parts.
✓ find the mid.

(A) $h \rightarrow 10 \rightarrow 20 \rightarrow$ | $n/2$
(B) $h \rightarrow 30 \rightarrow 20 \rightarrow 10$ | $n/2$

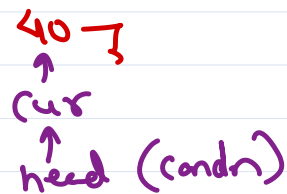
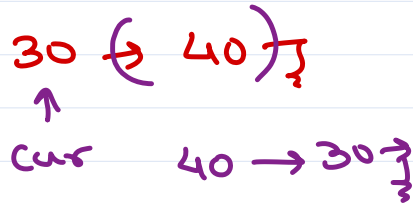
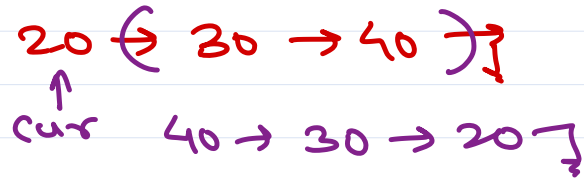
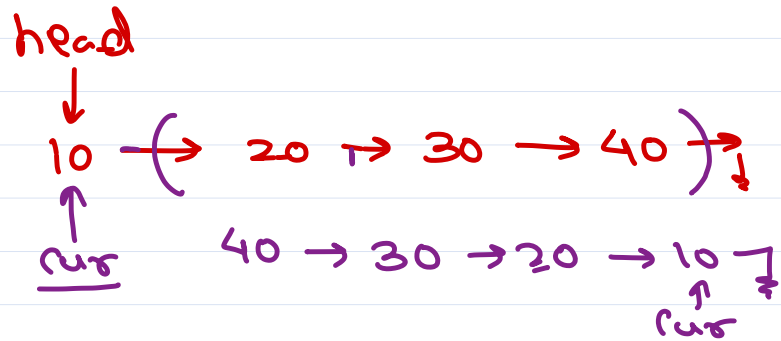
- ② reverse the second part: | $n/2$
(B) $h \rightarrow 10 \rightarrow 20 \rightarrow 30$ | $n/2$

- ③ Compare both parts. if they are same, list is palindrome, else not. | $n/2$
(A) == (B) + 30

$$itr = n - \frac{n}{2}$$



Reverse a Linked List using Recursion



```
Node reverse (Node cur) {
```

```
    if cur is last node, make it head & return;  
    if (cur.next == null) {
```

```
        head = cur;  
        return;
```

```
    }  
    reverse remaining list & return its tail (cur)
```

```
    tail = reverse (cur.next);
```

```
    add cur node to next of tail.
```

```
    tail.next = cur;
```

```
    make cur next as null;
```

```
    cur.next = null;
```

```
    return cur; // tail of reversed part
```

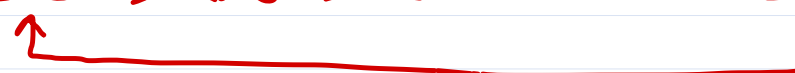
```
}
```

```
reverse (head);
```



Check if list has a loop

10 → 20 → 30 → 40 → 50 → 60 → 70 → 80 → 50



Approach 1: (Efficient)

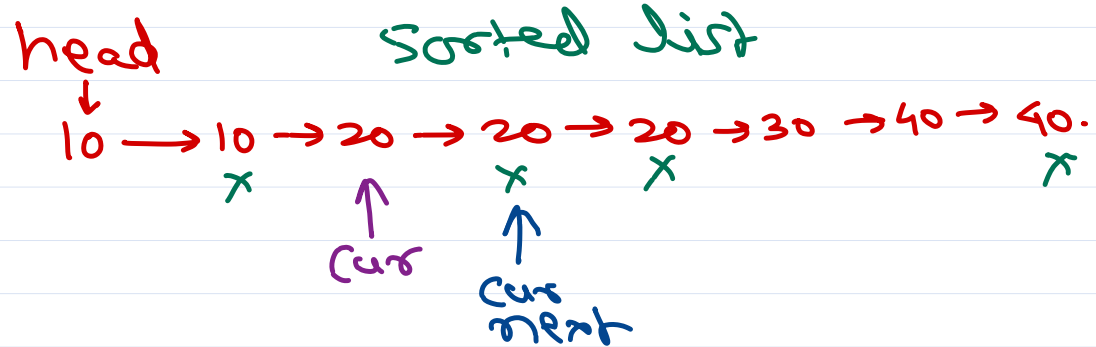
```
slow = head, fast = head;
while (fast != null && fast.next != null) {
    slow = slow.next;
    fast = fast.next.next;
    if (slow == fast)
        return true;
}
return false; // no cycle
```

Approach 2:

```
HashSet<Node> s = new HashSet<>();
trav = head;
while (trav != null) {
    if (s.add(trav) == false) {
        // if node is already added
        return true;
    }
    trav = trav.next;
}
return false; // no cycle
```

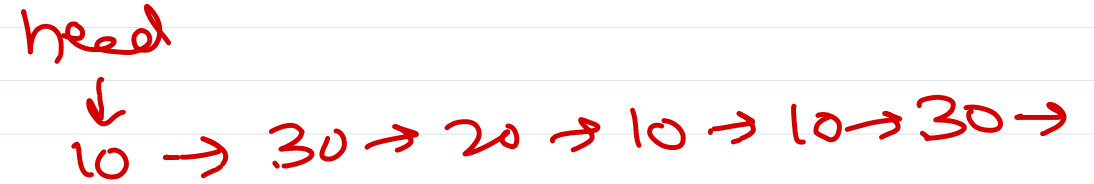


Delete duplicates from list



```
cur = head;
while (cur != null && cur.next != null) {
    while (cur.val == cur.next.val)
        cur.next = cur.next.next;
    cur = cur.next;
}
```

assignment: handle NullPointerException in above code.

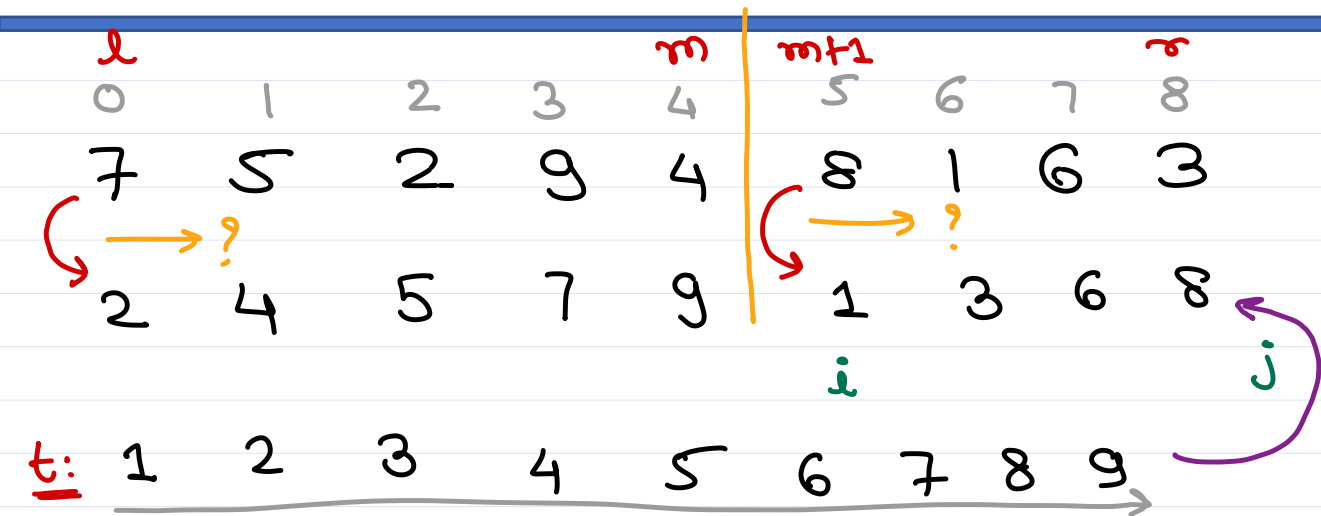


use HashSet to check if next value is already visited; if visited skip it (delete it).

```
Set<Integer> s = new HashSet(),
```



Merge Sort



```

t: 1 2 3 4 5 6 7 8 9
m = (l + r) / 2;
t = new int[r - l + 1];
i = l, j = m + 1, k = 0;
while (i <= m & j <= r) {
    if (a[i] < a[j])
        t[k++] = a[i++];
    else
        t[k++] = a[j++];
}
while (i <= m)
    t[k++] = a[i++];
while (j <= r)
    t[k++] = a[j++];
// now copy temp array to original array

```



Count occurrences of each character

```
str = "abcabcabc";
```

Approach 1:

brute-force approach.

- ✓ take each char and compare with remaining all (if not already visited).
- ✓ count all matching occurrences.

Approach 2: using hashtable key=char, val=count

```
HashMap<Character, Integer> map = new HashMap<>();
```

```
for (int i = 0; i < str.length(); i++) {  
    char ch = str.charAt(i);  
    int oldCnt = map.getOrDefault(ch, 0);  
    map.put(ch, oldCnt + 1);  
}
```

```
System.out.println(map);
```

Approach 3: without hash map.

```
arr = new int[128];  
// set all elem to zero (default)  
// in java.
```

```
for (i = 0; i < s.length(); i++) {  
    char ch = s.charAt(i);  
    arr[ch] = arr[ch] + 1;  
}
```

0	0
...	0
97	4
98	3
99	3
100	0
...	0
127	0

"abcabcabc"





Thank you!

Nilesh Ghule <nilesh@sunbeaminfo.com>

